



Etudiants :

El hadji Abdoulaye MBAYE

Ousmane BA

Mouhamed NDONGO

Mor NGOM

Alhousseynou AGNE

Mamadou NDIAYE

PROF : Mr Amadou NDIAYE

Module : P.O.O

**Conception
Et
Déploiement
D'un système de bibliothèque**

PLAN

Introduction

Objectifs

- I. Classe de base pour les utilisateurs**
- II. Classe pour les membres de la bibliothèque**
- III. Classe pour les employés de la bibliothèque**
- IV. Classe pour les livres**
- V. Classe pour les emprunts**
- VI. Gestion de la bibliothèque**
- VII. Interface utilisateur simple (console)**
- VIII. Classe principale pour lancer l'application**
- IX. Test de l'application**

Conclusion

Introduction

Les bibliothèques jouent un rôle fondamental dans la diffusion du savoir et de la culture. Toutefois, la gestion manuelle des livres, des membres et des emprunts peut s'avérer complexe et chronophage. C'est dans ce contexte que nous proposons le développement d'un **système de gestion de bibliothèque** informatisé, permettant d'optimiser l'organisation et la gestion des ressources documentaires. Grâce à une interface intuitive et des fonctionnalités avancées, cette application facilitera le suivi des emprunts et des retours, la gestion des membres, ainsi que l'administration des stocks de livres.

Objectifs

L'objectif principal de ce projet est de concevoir et de développer un système performant permettant la gestion efficace des livres et des membres d'une bibliothèque. Plus précisément, ce système vise :

- **Une Gestion optimisée des livres** : Ajouter, supprimer, modifier les informations et vérifier la disponibilité des ouvrages.
- **Une Gestion des membres** : Inscrire et consulter les informations des membres.
- **Un Suivi automatisé des emprunts** : Enregistrer les emprunts et retours en respectant les règles de prêt.
- **Une Gestion des rôles** : Différencier les permissions entre membres et employés, avec un système d'héritage.
- **Un Architecture logicielle robuste** : Définir des entités claires et appliquer les principes d'encapsulation pour sécuriser les données.

I. Classe de base pour les utilisateurs

```
1 package bibliotheque.utilisateurs;
2
3 public abstract class Personne {
4     private String nom;
5     private String prenom;
6     private String email;
7     private String telephone;
8     private String adresse;
9
10    public Personne(String nom, String prenom, String email, String telephone, String adresse) {
11        this.nom = nom;
12        this.prenom = prenom;
13        this.email = email;
14        this.telephone = telephone;
15        this.adresse = adresse;
16    }
17
18    // Getters et setters
19    public String getNom() {
20        return nom;
21    }
22
23    public void setNom(String nom) {
24        this.nom = nom;
25    }
26
27    public String getPrenom() {
28        return prenom;
29    }
30
31    public void setPrenom(String prenom) {
32        this.prenom = prenom;
33    }
34
35    public String getEmail() {
36        return email;
37    }
38
39    public void setEmail(String email) {
40        this.email = email;
41    }
42
43    public String getTelephone() {
44        return telephone;
45    }
46
47    public void setTelephone(String telephone) {
48        this.telephone = telephone;
49    }
50
51    public String getAdresse() {
52        return adresse;
53    }
54
55    public void setAdresse(String adresse) {
56        this.adresse = adresse;
57    }
58
59    @Override
60    public String toString() {
61        return prenom + " " + nom + " (" + email + ")";
62    }
63 }
```

Explication

La classe **Personne** est une **classe abstraite** qui sert de modèle pour les différentes personnes dans le système de la bibliothèque (comme

les membres et les employés). Elle contient des **attributs communs** et des **méthodes** pour gérer ces informations.

- ❖ Ce fichier appartient au **package bibliotheque.utilisateurs**, ce qui signifie qu'il est destiné à la gestion des utilisateurs (membres et employés).
- ❖ **abstract** signifie que cette classe ne peut pas être instanciée directement.
Elle est conçue pour être **héritée** par d'autres classes (comme Membre et Employe).
- ❖ Tous ces attributs sont **privés** (private), donc accessibles uniquement via des **méthodes publiques (getters/setters)**.
- ❖ Le constructeur avec **paramètre**, permet d'initialiser une Personne avec ses informations de base.
- ❖ Les méthodes **getters** permettant de retourner la valeur de la variable concernée et les méthodes **setters** de modifier la valeur de la variable concernée.
- ❖ **toString()** est une méthode spéciale qui retourne une représentation textuelle d'un objet. Ici, elle renvoie une chaîne sous la forme : *"Prénom Nom (email)"*.

II. Classe pour les membres de la bibliothèque

```

1 package bibliotheque.utilisateurs;
2
3 import java.util.ArrayList;
4
5 import java.util.Date;
6 import java.util.List;
7 import bibliotheque.livres.Emprunt;
8
9 public class Membre extends Personne {
10     private static int compteurId = 1;
11     private int id;
12     private Date dateInscription;
13     private List<Emprunt> emprunts;
14     private static final int MAX_EMPRUNTS = 3;
15
16     public Membre(String nom, String prenom, String email, String telephone, String adresse) {
17         super(nom, prenom, email, telephone, adresse);
18         this.id = compteurId++;
19         this.dateInscription = new Date();
20         this.emprunts = new ArrayList<>();
21     }
22
23     public int getId() {
24         return id;
25     }
26
27     public Date getDateInscription() {
28         return dateInscription;
29     }
30
31     public List<Emprunt> getEmprunts() {
32         return emprunts;
33     }
34
35     public boolean peutEmprunter() {
36         return emprunts.size() < MAX_EMPRUNTS;
37     }
38
39     public int getNombreEmprunts() {
40         return emprunts.size();
41     }
42
43     public void ajouterEmprunt(Emprunt emprunt) {
44         if (peutEmprunter()) {
45             emprunts.add(emprunt);
46         }
47     }
48
49     public void retirerEmprunt(Emprunt emprunt) {
50         emprunts.remove(emprunt);
51     }
52
53     @Override
54     public String toString() {
55         return "Membre #" + id + ": " + super.toString();
56     }
57 }

```

Explication

La classe Membre représente un **utilisateur** de la bibliothèque qui peut emprunter des livres. Elle **hérite** de la classe Personne et ajoute des fonctionnalités spécifiques aux membres.

- ❖ **package bibliotheque.utilisateurs;** → La classe Membre appartient au package bibliotheque.utilisateurs.
- ❖ **import java.util.Date;** → Pour gérer la date d'inscription du membre.
- ❖ **import java.util.List et import java.util.ArrayList;** → Pour stocker les emprunts sous forme de liste.
- ❖ **import bibliotheque.livres.Emprunt;** → Permet d'utiliser la classe Emprunt qui est définie dans un autre package.
- ❖ Membre **hérite** de Personne, ce qui signifie qu'il récupère tous ses attributs et méthodes (nom, prenom, etc.). Ajoute des fonctionnalités spécifiques aux membres de la bibliothèque.
- ❖ **Attributs:**
 - **compteurId** : Variable **statique** qui permet d'attribuer un **ID unique** à chaque membre.
 - **id** : Identifiant unique du membre.
 - **dateInscription** : Date à laquelle le membre s'est inscrit.
 - **emprunts** : Liste des livres empruntés par ce membre.
 - **MAX_EMPRUNTS** : Nombre maximum d'emprunts autorisés (fixé à 3).
- ❖ **Constructeur:**
 - **super(...)** → Appelle le **constructeur de Personne** pour initialiser nom, prenom, email, etc.
 - **this.id = compteurId++;** → Assigne un identifiant unique au membre et **incrémente** compteurId pour le prochain membre.
 - **this.dateInscription = new Date();** → Initialise la date d'inscription à la date actuelle.

- **this.emprunts = new ArrayList<>();** → Initialise la liste des emprunts.
- ❖ **Getters**, permettant d'accéder aux attributs privés.
- ❖ **Gestion des emprunts:**
 - **public boolean peutEmprunter()**
Vérifie si le membre peut emprunter un livre , retourne **true** si le membre a moins de MAX_EMPRUNTS (3). Sinon, retourne **false** (il ne peut plus emprunter).
 - **public int getNombreEmprunts()**

Obtenir le nombre d'emprunts, retourne le nombre de livres empruntés actuellement.
 - **public void ajouterEmprunt(Emprunt emprunt)**
Ajouter un emprunt, **vérifie d'abord** si le membre peut emprunter (peutEmprunter()). **Ajoute** l'emprunt à la liste si c'est possible.
 - **public void retirerEmprunt(Emprunt emprunt)**

Retirer un emprunt, **supprime** l'emprunt de la liste (lorsqu'un livre est retourné).
- ❖ **Redéfinition de toString()**, Ajoute **Membre #id:** devant la représentation de Personne.

III. Classe pour les employés de la bibliothèque


```

1 package bibliotheque.utilisateurs;
2
3 import java.util.Date;
4
5 public class Employe extends Personne {
6     private static int compteurId = 1;
7     private int id;
8     private Date dateEmbauche;
9     private String poste;
10
11     public Employe(String nom, String prenom, String email, String telephone, String adresse, String poste) {
12         super(nom, prenom, email, telephone, adresse);
13         this.id = compteurId++;
14         this.dateEmbauche = new Date();
15         this.poste = poste;
16     }
17
18     public int getId() {
19         return id;
20     }
21
22     public Date getDateEmbauche() {
23         return dateEmbauche;
24     }
25
26     public String getPoste() {
27         return poste;
28     }
29
30     public void setPoste(String poste) {
31         this.poste = poste;
32     }
33
34     @Override
35     public String toString() {
36         return "Employé #" + id + ": " + super.toString() + " - " + poste;
37     }
38 }

```

Explication

La classe Employe représente un employé de la bibliothèque. Elle **hérite** de la classe Personne et ajoute des fonctionnalités spécifiques aux employés, comme leur **poste** et leur **date d'embauche**.

- ❖ **package bibliotheque.utilisateurs;** → La classe Employe appartient au package bibliotheque.utilisateurs.
- ❖ **import java.util.Date;** → Permet de gérer la **date d'embauche** de l'employé.
- ❖ Employe **hérite** de Personne, ce qui signifie qu'il récupère tous ses attributs et méthodes (nom, prenom, email, etc.). Il ajoute des **attributs spécifiques aux employés**.
- ❖ **Attributs:**

- **compteurId** : Variable **statique** permettant d'attribuer un **ID unique** à chaque employé.
- **id** : Identifiant unique de l'employé.
- **dateEmbauche** : Date à laquelle l'employé a été embauché.
- **poste** : Le **poste** de l'employé (ex : bibliothécaire, archiviste, responsable...).

❖ **Constructeur**

- **super(...)** → Appelle le **constructeur de Personne** pour initialiser nom, prenom, email, etc.
- **this.id = compteurId++;** → Assigne un identifiant unique à l'employé et **incrémente** compteurId pour le prochain employé.
- **this.dateEmbauche = new Date();** → Initialise la date d'embauche à la **date actuelle**.
- **this.poste = poste;** → Initialise le **poste** de l'employé.

❖ **Getters et Setters**, ces méthodes permettent d'accéder aux attributs privés et de modifier le poste (**public void setPoste(String poste)**).

❖ **Redéfinition de toString()**, Ajoute **Employé #id:** devant la représentation de Personne. Ajoute aussi le poste à la fin.

IV. Classe pour les livres

```
1 package bibliotheque.livres;
2
3 public class Livre {
4     private String isbn;
5     private String titre;
6     private String auteur;
7     private String editeur;
8     private int anneePublication;
9     private int nombreExemplaires;
10    private int nombreExemplairesDisponibles;
11
12    public Livre(String isbn, String titre, String auteur, String editeur, int anneePublication, int nombreExemplaires) {
13        this.isbn = isbn;
14        this.titre = titre;
15        this.auteur = auteur;
16        this.editeur = editeur;
17        this.anneePublication = anneePublication;
18        this.nombreExemplaires = nombreExemplaires;
19        this.nombreExemplairesDisponibles = nombreExemplaires;
20    }
21
22    // Getters et setters
23    public String getIsbn() {
24        return isbn;
25    }
26
27    public String getTitre() {
28        return titre;
29    }
30
31    public void setTitre(String titre) {
32        this.titre = titre;
33    }
34
35    public String getAuteur() {
36        return auteur;
37    }
}
```

```

39● public void setAuteur(String auteur) {
40     this.auteur = auteur;
41 }
42
43● public String getEditeur() {
44     return editeur;
45 }
46
47● public void setEditeur(String editeur) {
48     this.editeur = editeur;
49 }
50
51● public int getAnneePublication() {
52     return anneePublication;
53 }
54
55● public void setAnneePublication(int anneePublication) {
56     this.anneePublication = anneePublication;
57 }
58
59● public int getNombreExemplaires() {
60     return nombreExemplaires;
61 }
62
63● public void setNombreExemplaires(int nombreExemplaires) {
64     int difference = nombreExemplaires - this.nombreExemplaires;
65     this.nombreExemplaires = nombreExemplaires;
66     this.nombreExemplairesDisponibles += difference;
67 }
68
69● public int getNombreExemplairesDisponibles() {
70     return nombreExemplairesDisponibles;
71 }
72
73● public boolean estDisponible() {
74     return nombreExemplairesDisponibles > 0;
75 }
76
77● public void emprunter() {
78     if (estDisponible()) {
79         nombreExemplairesDisponibles--;
80     }
81 }
82
83● public void retourner() {
84     if (nombreExemplairesDisponibles < nombreExemplaires) {
85         nombreExemplairesDisponibles++;
86     }
87 }
88
89● @Override
90 public String toString() {
91     return titre + " par " + auteur + " (ISBN: " + isbn + ") - " + nombreExemplairesDisponibles + "/" + nombreExemplaires + "
92 }

```

Explication

La classe Livre représente un **livre** dans la bibliothèque. Elle contient les **informations essentielles** sur un livre (titre, auteur, ISBN, éditeur, etc.) et gère le **nombre d'exemplaires disponibles** pour l'emprunt.

❖ Cette classe appartient au package **bibliotheque.livres**, ce qui permet de mieux organiser le code.

❖ **Attributs:**

- **isbn** → Code unique du livre (International Standard Book Number).
- **titre** → Titre du livre.
- **auteur** → Nom de l'auteur.
- **editeur** → Maison d'édition.
- **anneePublication** → Année de publication.
- **nombreExemplaires** → Nombre total d'exemplaires du livre dans la bibliothèque.
- **nombreExemplairesDisponibles** → Nombre d'exemplaires actuellement **disponibles** pour l'emprunt.

❖ **Constructeur:**

- **Initialise** tous les attributs du livre.
- Le **nombre d'exemplaires disponibles** est égal au **nombre total d'exemplaires** au départ.

❖ **Getters et Setters**

- Les **getters** permettent de récupérer les informations du livre.
- Les **setters** permettent de modifier les informations du livre.

❖ **Gestion du nombre d'exemplaires**

- **public void setNombreExemplaires(int nombreExemplaires)**

Modifier le nombre total d'exemplaires, Si on **augmente** le nombre total d'exemplaires, on met à jour le nombre d'exemplaires disponibles et si on **diminue**, cela réduit les exemplaires disponibles.

- **public boolean estDisponible()**

Vérifier si le livre est disponible, setourne **true** si au moins **un exemplaire** est disponible.

❖ **Gestion des emprunts et retours**

- **public void emprunter()**

Emprunter un livre, **diminue** le nombre d'exemplaires disponibles **si le livre est disponible**.

- **public void retourner()**

Retourner un livre, **augmente** le nombre d'exemplaires disponibles **si le livre n'a pas déjà atteint son stock total**.

❖ **Redéfinition de toString(), Affichage lisible** des informations du livre.

V. Classe pour les emprunts

```
1 package bibliotheque.livres;
2
3 import java.util.Date;
4
5 import java.util.Calendar;
6
7 import bibliotheque.utilisateurs.Membre;
8
9 public class Emprunt {
10     private static int compteurId = 1;
11     private int id;
12     private Livre livre;
13     private Membre membre;
14     private Date dateEmprunt;
15     private Date dateRetourPrevue;
16     private Date dateRetourEffective;
17     private static final int DUREE_EMPRUNT_JOURS = 14;
18
19     public Emprunt(Livre livre, Membre membre) {
20         this.id = compteurId++;
21         this.livre = livre;
22         this.membre = membre;
23         this.dateEmprunt = new Date();
24
25         // Calcul de la date de retour prévue (date d'emprunt + durée d'emprunt)
26         Calendar calendar = Calendar.getInstance();
27         calendar.setTime(dateEmprunt);
28         calendar.add(Calendar.DAY_OF_MONTH, DUREE_EMPRUNT_JOURS);
29         this.dateRetourPrevue = calendar.getTime();
30
31         this.dateRetourEffective = null; // Pas encore retourné
32
33         // Mise à jour des objets liés
34         livre.emprunter();
35         membre.ajouterEmprunt(this);
36     }
}
```

```
38● public int getId() {
39     return id;
40 }
41
42● public Livre getLivre() {
43     return livre;
44 }
45
46● public Membre getMembre() {
47     return membre;
48 }
49
50● public Date getDateEmprunt() {
51     return dateEmprunt;
52 }
53
54● public Date getDateRetourPrevue() {
55     return dateRetourPrevue;
56 }
57
58● public Date getDateRetourEffective() {
59     return dateRetourEffective;
60 }
61
62● public boolean estRetourne() {
63     return dateRetourEffective != null;
64 }
65
66● public boolean estEnRetard() {
67     if (estRetourne()) {
68         return false;
69     }
70     return new Date().after(dateRetourPrevue);
71 }
72
```



```

73 public void retourner() {
74     if (!estRetourne()) {
75         this.dateRetourEffective = new Date();
76         livre.retourner();
77         membre.retirerEmprunt(this);
78     }
79 }
80
81 @Override
82 public String toString() {
83     String status = estRetourne() ? "Retourné le " + dateRetourEffective : "À retourner avant le " + dateRetourPrevue;
84     return "Emprunt #" + id + ": " + livre.getTitre() + " par " + membre.getPrenom() + " " + membre.getNom() + " - " + status;
85 }
86 }

```

Explication

La classe Emprunt représente un **emprunt de livre** par un membre de la bibliothèque. Elle permet de gérer les informations sur :

- Le **livre emprunté**.
- Le **membre emprunteur**.
- Les **dates d'emprunt et de retour**.
- L'**état de l'emprunt** (en cours, retourné, en retard).
- ❖ **package bibliotheque.livres;** → Indique que cette classe appartient au package **livres**.
- ❖ **import java.util.Date;** → Utilisé pour gérer les dates d'emprunt et de retour.
- ❖ **import java.util.Calendar;** → Permet de calculer la date de retour prévue.
- ❖ **import bibliotheque.utilisateurs.Membre;** → Permet d'associer un emprunt à un membre.
- ❖ **Attributs:**
 - **compteurId** → Gère l'identifiant unique des emprunts.
 - **id** → Identifiant unique de l'emprunt.
 - **livre** → Le livre emprunté (objet de type Livre).
 - **membre** → Le membre qui emprunte le livre.
 - **dateEmprunt** → La date à laquelle le livre a été emprunté.

- **dateRetourPrevue** → La date à laquelle le livre doit être retourné.
- **dateRetourEffective** → La date à laquelle le livre a été réellement retourné (initialement null).
- **DUREE_EMPRUNT_JOURS** → Durée standard d'un emprunt en jours (**14 jours**).

❖ **Constructeur : public Emprunt (Livre livre, Membre membre)**

- **Génère un ID unique** pour chaque emprunt (compteurId++).
- **Stocke le livre et le membre** liés à l'emprunt.
- **Définit la date d'emprunt** à la date actuelle.
- **Calcule la date de retour prévue** en ajoutant 14 jours à la date d'emprunt.
- **Initialise dateRetourEffective à null**, car le livre n'a pas encore été rendu.
- **Met à jour les objets Livre et Membre :**
 - **livre.emprunter();** → Diminue le nombre d'exemplaires disponibles du livre.
 - **membre.ajouterEmprunt(this);** → Ajoute cet emprunt à la liste des emprunts du membre.

❖ **Getters**

Elles permettent de :

- Obtenir l'**ID de l'emprunt**.
- Accéder au **livre** et au **membre** concernés.
- Consulter les **dates d'emprunt et de retour**.

❖ **Vérifications de l'état de l'emprunt**

- **public boolean estRetourne()**
Vérifier si le livre a été retourné, Retourne **true** si le livre a été retourné (**dateRetourEffective n'est plus null**).
- **public boolean estEnRetard()**
Vérifier si l'emprunt est en retard, Un emprunt est **en retard** si la date actuelle est **postérieure à la date de retour prévue**.
Si le livre est déjà retourné, il n'est **pas en retard**.

❖ Gestion du retour du livre

public void retourner()

- Vérifie si le livre n'a **pas encore été retourné**.
- **Enregistre la date de retour** (dateRetourEffective).
- **Augmente le nombre d'exemplaires disponibles** du livre (livre.retourner();).
- **Supprime l'emprunt de la liste des emprunts actifs du membre** (membre.retirerEmprunt(this);).

❖ Redéfinition de **toString()**, affichage de l'état d'emprunt si le livre est retourné ou doit être retourné.

VI. Gestion de la bibliothèque

```
1 package bibliotheque.gestion;
2
3 import java.util.ArrayList;
4
5
6 import java.util.HashMap;
7 import java.util.List;
8 import java.util.Map;
9
10 import bibliotheque.livres.Emprunt;
11 import bibliotheque.livres.Livre;
12 import bibliotheque.utilisateurs.Employe;
13 import bibliotheque.utilisateurs.Membre;
14
15 public class Bibliotheque {
16     private Map<String, Livre> livres; // Map ISBN -> Livre
17     private Map<Integer, Membre> membres; // Map ID -> Membre
18     private Map<Integer, Employe> employes; // Map ID -> Employé
19     private List<Emprunt> emprunts; // Liste de tous les emprunts
```

```
20 public Bibliotheque() {
21     this.livres = new HashMap<>();
22     this.membres = new HashMap<>();
23     this.employes = new HashMap<>();
24     this.emprunts = new ArrayList<>();
25 }
26
27 // Gestion des livres
28 public void ajouterLivre(Livre livre) {
29     livres.put(livre.getIsbn(), livre);
30 }
31
32 public void supprimerLivre(String isbn) {
33     livres.remove(isbn);
34 }
35
36 public Livre rechercherLivre(String isbn) {
37     return livres.get(isbn);
38 }
39
40 public List<Livre> getTousLesLivres() {
41     return new ArrayList<>(livres.values());
42 }
43
44 public List<Livre> rechercherLivresParTitre(String titre) {
45     List<Livre> resultats = new ArrayList<>();
46     for (Livre livre : livres.values()) {
47         if (livre.getTitre().toLowerCase().contains(titre.toLowerCase())) {
48             resultats.add(livre);
49         }
50     }
51     return resultats;
52 }
```

```

54● public List<Livres> rechercherLivresParAuteur(String auteur) {
55     List<Livres> resultats = new ArrayList<>();
56     for (Livres livre : livres.values()) {
57         if (livre.getAuteur().toLowerCase().contains(auteur.toLowerCase())) {
58             resultats.add(livre);
59         }
60     }
61     return resultats;
62 }
63
64 // Gestion des membres
65● public void inscrireMembre(Membre membre) {
66     membres.put(membre.getId(), membre);
67 }
68
69● public void supprimerMembre(int id) {
70     membres.remove(id);
71 }
72
73● public Membre rechercherMembre(int id) {
74     return membres.get(id);
75 }
76
77● public List<Membre> getTousLesMembres() {
78     return new ArrayList<>(membres.values());
79 }
80
81 // Gestion des employés
82● public void ajouterEmploye(Employe employe) {
83     employes.put(employe.getId(), employe);
84 }
85
86● public void supprimerEmploye(int id) {
87     employes.remove(id);
88 }
89
90● public Employe rechercherEmploye(int id) {
91     return employes.get(id);
92 }
93
94● public List<Employe> getTousLesEmployes() {
95     return new ArrayList<>(employes.values());
96 }
97

```

```

98      // Gestion des emprunts
99  public Emprunt emprunterLivres(String isbn, int membreId) {
100      Livre livre = livres.get(isbn);
101      Membre membre = membres.get(membreId);
102
103      if (livre == null || membre == null) {
104          return null;
105      }
106
107      if (!livre.estDisponible()) {
108          return null;
109      }
110
111      if (!membre.peutEmprunter()) {
112          return null;
113      }
114
115      Emprunt emprunt = new Emprunt(livre, membre);
116      emprunts.add(emprunt);
117
118      return emprunt;
119  }
120
121  public boolean retournerLivres(int empruntId) {
122      for (Emprunt emprunt : emprunts) {
123          if (emprunt.getId() == empruntId && !emprunt.estRetourne()) {
124              emprunt.retourner();
125              return true;
126          }
127      }
128      return false;
129  }
130
131  public List<Emprunt> getTousLesEmprunts() {
132      return new ArrayList<>(emprunts);
133  }
134

```

```

135  public List<Emprunt> getEmpruntsEnCours() {
136      List<Emprunt> empruntsEnCours = new ArrayList<>();
137      for (Emprunt emprunt : emprunts) {
138          if (!emprunt.estRetourne()) {
139              empruntsEnCours.add(emprunt);
140          }
141      }
142      return empruntsEnCours;
143  }
144
145  public List<Emprunt> getEmpruntsEnRetard() {
146      List<Emprunt> empruntsEnRetard = new ArrayList<>();
147      for (Emprunt emprunt : emprunts) {
148          if (!emprunt.estRetourne() && emprunt.estEnRetard()) {
149              empruntsEnRetard.add(emprunt);
150          }
151      }
152      return empruntsEnRetard;
153  }
154 }

```

Explication

La classe **Bibliotheque** est le **gestionnaire central** de la bibliothèque. Elle permet de gérer :

- **Les livres** (ajout, suppression, recherche).
- **Les membres** (inscription, suppression, recherche).
- **Les employés** (ajout, suppression, recherche).
- **Les emprunts** (emprunter, retourner un livre, lister les emprunts en cours et en retard).
- ❖ **java.util.HashMap et java.util.Map** : Ces classes sont utilisées pour stocker des livres, membres et employés dans des **dictionnaires** où les clés sont respectivement l'ISBN des livres et les IDs des membres/employés.
- ❖ **java.util.List** : Utilisée pour stocker des listes, comme les emprunts d'un membre.
- ❖ **bibliotheque.livres.Emprunt et bibliotheque.livres.Livre** : Ces classes représentent les livres et les emprunts, respectivement.
- ❖ **bibliotheque.utilisateurs.Employe et bibliotheque.utilisateurs.Membre** : Ces classes représentent les employés et les membres de la bibliothèque.
- ❖ **Attributs** :
 - **livres** → Stocke les livres de la bibliothèque (clé = ISBN, valeur = Livre).
 - **membres** → Stocke les membres inscrits (clé = ID, valeur = Membre).
 - **employes** → Stocke les employés de la bibliothèque (clé = ID, valeur = Employe).
 - **emprunts** → Liste contenant tous les emprunts effectués.
- ❖ **Constructeur**
Initialise les structures de stockage (**HashMap et ArrayList**).
- ❖ **Gestion des livres**
 - **public void ajouterLivre(Livre livre)**
Ajouter un livre, ajoute un livre à la bibliothèque en l'associant à son ISBN.

- **public void supprimerLivre(String isbn)**
Supprimer un livre, Supprime un livre de la bibliothèque en utilisant son ISBN.
- **public void supprimerLivre(String isbn)**
Retourne le livre correspondant à l'ISBN fourni.
- **public List<Livre> getAllLivres()**
Retourne une **liste** de tous les livres de la bibliothèque.
- **public List<Livre> rechercherLivresParTitre(String titre)**
Retourne une liste des livres dont le **titre contient** le texte fourni.
- **public List<Livre> rechercherLivresParAuteur(String auteur)**
Retourne une liste des livres écrits par l'auteur recherché.

❖ Gestion des membres

- **public void inscrireMembre(Membre membre)**
Ajoute un nouveau membre à la bibliothèque.
- **public void supprimerMembre(int id)**
Supprime un membre en fonction de son ID.
- **public Membre rechercherMembre(int id)**
Retourne le membre correspondant à l'ID fourni.
- **public List<Membre> getAllMembres()**
Retourne une **liste** de tous les membres.

❖ Gestion des employés

- **public void ajouterEmploye(Employe employe)**
Ajoute un employé à la bibliothèque.
- **public void supprimerEmploye(int id)**
Supprime un employé en fonction de son ID.
- **public Employe rechercherEmploye(int id)**
Retourne l'employé correspondant à l'ID fourni.
- **public List<Employe> getAllEmployes()**
Retourne une **liste** de tous les employés.

❖ Gestion des emprunts

- **public Emprunt emprunterLivre(String isbn, int membreId)**
 - Vérifie si le **livre et le membre existent**.
 - Vérifie si le **livre est disponible**.
 - Vérifie si le **membre a le droit d'emprunter**.
 - **Crée un emprunt** et l'ajoute à la liste des emprunts.
- **public boolean retournerLivre(int empruntId)**
 - Recherche l'emprunt correspondant à empruntId.
 - Vérifie si l'emprunt **n'a pas encore été retourné**.
 - **Retourne le livre** et met à jour les stocks.
- **public List<Emprunt> getAllEmprunts()**
Retourne la **liste complète** des emprunts.
- **public List<Emprunt> getAllEmpruntsEnCours()**
Retourne les **emprunts qui n'ont pas encore été retournés**.
- **public List<Emprunt> getAllEmpruntsEnRetard()**
Retourne les **emprunts qui sont en retard**.

VII. Interface utilisateur simple (console)

```

1 package bibliotheque.ui;
2
3 import java.util.List;
4 import java.util.Scanner;
5 import bibliotheque.gestion.Bibliotheque;
6 import bibliotheque.livres.Emprunt;
7 import bibliotheque.livres.Livre;
8 import bibliotheque.utilisateurs.Employe;
9 import bibliotheque.utilisateurs.Membre;
10
11 public class InterfaceConsole {
12     private Bibliotheque bibliotheque;
13     private Scanner scanner;
14     private boolean estEmploye;
15     private int utilisateurId;
16
17     public InterfaceConsole() {
18         this.bibliotheque = new Bibliotheque();
19         this.scanner = new Scanner(System.in);
20         this.estEmploye = false;
21         this.utilisateurId = -1;
22
23         // Ajouter des données de test
24         initialiserDonneesTest();
25     }
26
27     private void initialiserDonneesTest() {
28         // Ajouter quelques livres
29         bibliotheque.ajouterLivre(new Livre("978-2-7654-1005-3", "Le Petit Prince", "Antoine de Saint-Exupéry", "Gallimard", 1943, 5));
30         bibliotheque.ajouterLivre(new Livre("978-2-07-036822-8", "L'Étranger", "Albert Camus", "Gallimard", 1942, 3));
31         bibliotheque.ajouterLivre(new Livre("978-2-253-06541-0", "1984", "George Orwell", "Gallimard", 1949, 2));
32
33         // Ajouter des membres
34         bibliotheque.inscrireMembre(new Membre("Dupont", "Jean", "jean.dupont@email.com", "0601020304", "123 Rue de Paris"));
35         bibliotheque.inscrireMembre(new Membre("Martin", "Sophie", "sophie.martin@email.com", "0602030405", "456 Avenue des Fleurs"));
36
37         // Ajouter des employés
38         bibliotheque.ajouterEmploye(new Employe("Dubois", "Michel", "michel.dubois@biblio.fr", "0701020304", "789 Boulevard Central",
39             "Bibliothécaire"));
40     }
41
42     public void demarrer() {
43         boolean continuer = true;
44
45         System.out.println("=== SYSTÈME DE GESTION DE BIBLIOTHÈQUE ===");
46
47         while (continuer) {
48             if (utilisateurId == -1) {
49                 authentication();
50             } else if (estEmploye) {
51                 menuEmploye();
52             } else {
53                 menuMembre();
54             }
55
56             System.out.println("\nVoulez-vous continuer ? (O/N)");
57             String reponse = scanner.nextLine().trim().toUpperCase();
58             if (!reponse.equals("O")) {
59                 continuer = false;
60             }
61         }
62
63         System.out.println("Au revoir !");
64         scanner.close();
65     }

```

```

67 private void authentification() {
68     System.out.println("\n=== AUTHENTIFICATION ===");
69     System.out.println("1. Se connecter en tant que membre");
70     System.out.println("2. Se connecter en tant qu'employé");
71     System.out.print("Votre choix : ");
72
73     int choix = Integer.parseInt(scanner.nextLine());
74
75     if (choix == 1) {
76         // Connexion membre
77         estEmploye = false;
78
79         System.out.print("Entrez votre ID membre : ");
80         int id = Integer.parseInt(scanner.nextLine());
81
82         Membre membre = bibliotheque.rechercherMembre(id);
83         if (membre != null) {
84             utilisateurId = id;
85             System.out.println("Bienvenue, " + membre.getPrenom() + " " + membre.getNom() + " !");
86         } else {
87             System.out.println("Membre non trouvé.");
88         }
89     } else if (choix == 2) {
90         // Connexion employé
91         estEmploye = true;
92
93         System.out.print("Entrez votre ID employé : ");
94         int id = Integer.parseInt(scanner.nextLine());
95
96         Employe employe = bibliotheque.rechercherEmploye(id);
97         if (employe != null) {
98             utilisateurId = id;
99             System.out.println("Bienvenue, " + employe.getPrenom() + " " + employe.getNom() + " !");
100         } else {
101             System.out.println("Employé non trouvé.");
102         }
103     } else {
104         System.out.println("Choix invalide.");
105     }
106 }

```

```

108 private void menuMembre() {
109     Membre membre = bibliotheque.rechercherMembre(utilisateurId);
110
111     System.out.println("\n=== MENU MEMBRE ===");
112     System.out.println("1. Consulter les livres disponibles");
113     System.out.println("2. Rechercher un livre par titre");
114     System.out.println("3. Rechercher un livre par auteur");
115     System.out.println("4. Consulter mes emprunts");
116     System.out.println("5. Se déconnecter");
117     System.out.print("Votre choix : ");
118
119     int choix = Integer.parseInt(scanner.nextLine());
120
121     switch (choix) {
122         case 1:
123             // Consulter les livres disponibles
124             afficherLivres(bibliotheque.getTousLesLivres());
125             break;
126         case 2:
127             // Rechercher un livre par titre
128             System.out.print("Entrez le titre à rechercher : ");
129             String titre = scanner.nextLine();
130             afficherLivres(bibliotheque.rechercherLivresParTitre(titre));
131             break;
132         case 3:
133             // Rechercher un livre par auteur
134             System.out.print("Entrez l'auteur à rechercher : ");
135             String auteur = scanner.nextLine();
136             afficherLivres(bibliotheque.rechercherLivresParAuteur(auteur));
137             break;
138         case 4:
139             // Consulter mes emprunts
140             System.out.println("\n=== MES EMPRUNTS ===");
141             for (Emprunt emprunt : membre.getEmprunts()) {
142                 System.out.println(emprunt);
143             }
144             break;

```

```

145         case 5:
146             // Se déconnecter
147             utilisateurId = -1;
148             estEmploye = false;
149             System.out.println("Vous êtes déconnecté.");
150             break;
151         default:
152             System.out.println("Choix invalide.");
153     }
154 }
155
156 private void menuEmploye() {
157     System.out.println("\n=== MENU EMPLOYÉ ===");
158     System.out.println("1. Gérer les livres");
159     System.out.println("2. Gérer les membres");
160     System.out.println("3. Gérer les emprunts");
161     System.out.println("4. Se déconnecter");
162     System.out.print("Votre choix : ");
163
164     int choix = Integer.parseInt(scanner.nextLine());
165
166     switch (choix) {
167         case 1:
168             menuGestionLivres();
169             break;
170         case 2:
171             menuGestionMembres();
172             break;
173         case 3:
174             menuGestionEmprunts();
175             break;
176         case 4:
177             utilisateurId = -1;
178             estEmploye = false;
179             System.out.println("Vous êtes déconnecté.");
180             break;
181         default:
182             System.out.println("Choix invalide.");
183     }
184 }

```

```

186 private void menuGestionLivres() {
187     System.out.println("\n=== GESTION DES LIVRES ===");
188     System.out.println("1. Ajouter un livre");
189     System.out.println("2. Supprimer un livre");
190     System.out.println("3. Mettre à jour un livre");
191     System.out.println("4. Consulter tous les livres");
192     System.out.println("5. Rechercher un livre par ISBN");
193     System.out.print("Votre choix : ");
194
195     int choix = Integer.parseInt(scanner.nextLine());
196
197     switch (choix) {
198         case 1:
199             // Ajouter un livre
200             System.out.print("ISBN : ");
201             String isbn = scanner.nextLine();
202             System.out.print("Titre : ");
203             String titre = scanner.nextLine();
204             System.out.print("Auteur : ");
205             String auteur = scanner.nextLine();
206             System.out.print("Éditeur : ");
207             String editeur = scanner.nextLine();
208             System.out.print("Année de publication : ");
209             int annee = Integer.parseInt(scanner.nextLine());
210             System.out.print("Nombre d'exemplaires : ");
211             int nombre = Integer.parseInt(scanner.nextLine());
212
213             Livre nouveauLivre = new Livre(isbn, titre, auteur, editeur, annee, nombre);
214             bibliotheque.ajouterLivre(nouveauLivre);
215             System.out.println("Livre ajouté avec succès !");
216             break;
217         case 2:
218             // Supprimer un livre
219             System.out.print("ISBN du livre à supprimer : ");
220             String isbnSupp = scanner.nextLine();
221             bibliotheque.supprimerLivre(isbnSupp);
222             System.out.println("Livre supprimé !");
223             break;
224         case 3:
225             // Mettre à jour un livre
226             System.out.print("ISBN du livre à mettre à jour : ");
227             String isbnMaj = scanner.nextLine();
228             Livre livre = bibliotheque.rechercherLivre(isbnMaj);
229
230             if (livre != null) {
231                 System.out.println("Livre trouvé : " + livre);
232                 System.out.println("Laissez vide pour conserver les valeurs actuelles.");
233
234                 System.out.print("Nouveau titre (" + livre.getTitre() + ") : ");
235                 String nouveauTitre = scanner.nextLine();
236                 if (!nouveauTitre.isEmpty()) {
237                     livre.setTitre(nouveauTitre);
238                 }
239
240                 System.out.print("Nouvel auteur (" + livre.getAuteur() + ") : ");
241                 String nouvelAuteur = scanner.nextLine();
242                 if (!nouvelAuteur.isEmpty()) {
243                     livre.setAuteur(nouvelAuteur);
244                 }
245
246                 System.out.print("Nouvel éditeur (" + livre.getEditeur() + ") : ");
247                 String nouvelEditeur = scanner.nextLine();
248                 if (!nouvelEditeur.isEmpty()) {
249                     livre.setEditeur(nouvelEditeur);
250                 }
251
252                 System.out.print("Nouvelle année (" + livre.getAnneePublication() + ") : ");
253                 String nouvelleAnnee = scanner.nextLine();
254                 if (!nouvelleAnnee.isEmpty()) {
255                     livre.setAnneePublication(Integer.parseInt(nouvelleAnnee));
256                 }
257
258                 System.out.print("Nouveau nombre d'exemplaires (" + livre.getNombreExemplaires() + ") : ");
259                 String nouveauNombre = scanner.nextLine();
260                 if (!nouveauNombre.isEmpty()) {
261                     livre.setNombreExemplaires(Integer.parseInt(nouveauNombre));
262                 }
263             }

```

```

264         System.out.println("Livres mis à jour avec succès !");
265     } else {
266         System.out.println("Livres non trouvés.");
267     }
268     break;
269 case 4:
270     // Consulter tous les livres
271     afficherLivres(bibliotheque.getTousLesLivres());
272     break;
273 case 5:
274     // Rechercher un livre par ISBN
275     System.out.print("ISBN : ");
276     String isbnRecherche = scanner.nextLine();
277     Livre livreRecherche = bibliotheque.rechercherLivre(isbnRecherche);
278
279     if (livreRecherche != null) {
280         System.out.println("Livre trouvé : " + livreRecherche);
281     } else {
282         System.out.println("Livre non trouvé.");
283     }
284     break;
285 default:
286     System.out.println("Choix invalide.");
287 }
288 }
289

```

```

290 private void menuGestionMembres() {
291     System.out.println("\n=== GESTION DES MEMBRES ===");
292     System.out.println("1. Inscrire un nouveau membre");
293     System.out.println("2. Supprimer un membre");
294     System.out.println("3. Consulter tous les membres");
295     System.out.println("4. Rechercher un membre par ID");
296     System.out.print("Votre choix : ");
297
298     int choix = Integer.parseInt(scanner.nextLine());
299
300     switch (choix) {
301     case 1:
302         // Inscrire un nouveau membre
303         System.out.print("Nom : ");
304         String nom = scanner.nextLine();
305         System.out.print("Prénom : ");
306         String prenom = scanner.nextLine();
307         System.out.print("Email : ");
308         String email = scanner.nextLine();
309         System.out.print("Téléphone : ");
310         String telephone = scanner.nextLine();
311         System.out.print("Adresse : ");
312         String adresse = scanner.nextLine();
313
314         Membre nouveauMembre = new Membre(nom, prenom, email, telephone, adresse);
315         bibliotheque.inscrireMembre(nouveauMembre);
316         System.out.println("Membre inscrit avec succès ! ID : " + nouveauMembre.getId());
317         break;
318     case 2:
319         // Supprimer un membre
320         System.out.print("ID du membre à supprimer : ");
321         int idSupp = Integer.parseInt(scanner.nextLine());
322         bibliotheque.supprimerMembre(idSupp);
323         System.out.println("Membre supprimé !");
324         break;

```



```

325         case 3:
326             // Consulter tous les membres
327             System.out.println("\n=== LISTE DES MEMBRES ===");
328             for (Membre membre : bibliotheque.getTousLesMembres()) {
329                 System.out.println(membre);
330             }
331             break;
332         case 4:
333             // Rechercher un membre par ID
334             System.out.print("ID du membre : ");
335             int idRecherche = Integer.parseInt(scanner.nextLine());
336             Membre membreRecherche = bibliotheque.rechercherMembre(idRecherche);
337
338             if (membreRecherche != null) {
339                 System.out.println("Membre trouvé : " + membreRecherche);
340                 System.out.println("Nombre d'emprunts en cours : " + membreRecherche.getNombreEmprunts());
341             } else {
342                 System.out.println("Membre non trouvé.");
343             }
344             break;
345         default:
346             System.out.println("Choix invalide.");
347     }
348 }
349

```

```

350 private void menuGestionEmprunts() {
351     System.out.println("\n=== GESTION DES EMPRUNTS ===");
352     System.out.println("1. Enregistrer un nouvel emprunt");
353     System.out.println("2. Enregistrer un retour");
354     System.out.println("3. Consulter tous les emprunts");
355     System.out.println("4. Consulter les emprunts en cours");
356     System.out.println("5. Consulter les emprunts en retard");
357     System.out.print("Votre choix : ");
358
359     int choix = Integer.parseInt(scanner.nextLine());
360
361     switch (choix) {
362         case 1:
363             // Enregistrer un nouvel emprunt
364             System.out.print("ID du membre : ");
365             int idMembre = Integer.parseInt(scanner.nextLine());
366             System.out.println("ISBN du livre : ");
367             String isbn = scanner.nextLine();
368
369             Emprunt nouvelEmprunt = bibliotheque.emprunterLivre(isbn, idMembre);
370
371             if (nouvelEmprunt != null) {
372                 System.out.println("Emprunt enregistré avec succès ! ID : " + nouvelEmprunt.getId());
373             } else {
374                 System.out.println("Impossible d'enregistrer l'emprunt. Vérifiez que :");
375                 System.out.println("- Le membre et le livre existent");
376                 System.out.println("- Le livre est disponible");
377                 System.out.println("- Le membre n'a pas atteint sa limite d'emprunts");
378             }
379             break;

```



```

380         case 2:
381             // Enregistrer un retour
382             System.out.print("ID de l'emprunt : ");
383             int idEmprunt = Integer.parseInt(scanner.nextLine());
384
385             boolean retourReussi = bibliotheque.retournerLivre(idEmprunt);
386
387             if (retourReussi) {
388                 System.out.println("Retour enregistré avec succès !");
389             } else {
390                 System.out.println("Impossible d'enregistrer le retour. Vérifiez que :");
391                 System.out.println("- L'emprunt existe");
392                 System.out.println("- Le livre n'a pas déjà été retourné");
393             }
394             break;
395         case 3:
396             // Consulter tous les emprunts
397             System.out.println("\n=== LISTE DE TOUS LES EMPRUNTS ===");
398             for (Emprunt emprunt : bibliotheque.getTousLesEmprunts()) {
399                 System.out.println(emprunt);
400             }
401             break;
402         case 4:
403             // Consulter les emprunts en cours
404             System.out.println("\n=== LISTE DES EMPRUNTS EN COURS ===");
405             for (Emprunt emprunt : bibliotheque.getEmpruntsEnCours()) {
406                 System.out.println(emprunt);
407             }
408             break;
409         case 5:
410             // Consulter les emprunts en retard
411             System.out.println("\n=== LISTE DES EMPRUNTS EN RETARD ===");
412             for (Emprunt emprunt : bibliotheque.getEmpruntsEnRetard()) {
413                 System.out.println(emprunt);
414             }
415             break;
416         default:
417             System.out.println("Choix invalide.");
418     }
419 }

```

```

420
421 private void afficherLivres(List<Livres> livres) {
422     System.out.println("\n=== LISTE DES LIVRES ===");
423     if (livres.isEmpty()) {
424         System.out.println("Aucun livre trouvé.");
425     } else {
426         for (Livres livre : livres) {
427             System.out.println(livre);
428         }
429     }
430 }
431 }

```

Explication

Ce code représente une interface utilisateur en ligne de commande pour gérer une bibliothèque. Il permet aux utilisateurs de s'authentifier en tant que **membre** ou **employé** et d'effectuer diverses opérations, comme **l'emprunt de livres**, **la gestion des membres** et **l'administration des livres**.

- ❖ **bibliotheque.gestion.Bibliotheque** : Gère l'ensemble des livres, membres et employés.
- ❖ **bibliotheque.livres.Livre** : Représente un livre.
- ❖ **bibliotheque.livres.Emprunt** : Gère les emprunts de livres.
- ❖ **bibliotheque.utilisateurs.Membre** : Représente un utilisateur de la bibliothèque qui peut emprunter des livres.
- ❖ **bibliotheque.utilisateurs.Employe** : Représente un employé qui administre la bibliothèque.
- ❖ L'interface console utilise un **scanner** (Scanner scanner) pour interagir avec l'utilisateur.

❖ Initialisation des Données

La Méthode **initialiserDonneesTest()** ajoute des **livres**, des **membres** et un **employé** à la bibliothèque lors du démarrage, facilitant les tests :

- **Ajout de livres** avec des ISBN, auteurs et maisons d'édition.
- **Création de membres** avec des informations personnelles.
- **Ajout d'un employé** qui pourra gérer les ressources de la bibliothèque.

❖ Authentification

Avec la Méthode **authentification()**, L'utilisateur choisit entre:

- **Se connecter en tant que membre** → Il entre son ID membre.
 - S'il existe, il accède au **menu membre**.
 - Sinon, un message d'erreur s'affiche.
- **Se connecter en tant qu'employé** → Il entre son ID employé.
 - S'il est reconnu, il accède au **menu employé**.
 - Sinon, un message d'erreur s'affiche.

❖ Menu Membre

Accessible après authentification réussie, il propose plusieurs options :

- **Consulter les livres**

- **Afficher tous les livres disponibles** (getTousLesLivres()).
- **Rechercher un livre par titre**
(rechercherLivresParTitre(String titre)).
- **Rechercher un livre par auteur**
(rechercherLivresParAuteur(String auteur)).

- **Consulter ses emprunts**

Affiche la liste des emprunts du membre
(membre.getEmprunts()).

- **Se déconnecter**

- Réinitialise les variables utilisateurId et estEmploye pour permettre une nouvelle authentification.

❖ Menu Employé

L'employé a accès à des fonctionnalités avancées :

- **Gérer les livres**

- **Ajouter un livre** (ajouterLivre(Livre livre)).
- **Supprimer un livre** (supprimerLivre(String isbn)).
- **Mettre à jour un livre** en modifiant ses attributs.
- **Rechercher un livre par ISBN** (rechercherLivre(String isbn)).

- **Gérer les membres**

- **Inscrire un nouveau membre** (inscrireMembre(Membre membre)).
- **Supprimer un membre** (supprimerMembre(int id)).
- **Afficher la liste des membres** (getTousLesMembres()).
- **Rechercher un membre par ID** (rechercherMembre(int id)).

- **Gérer les emprunts**

- **Enregistrer un emprunt** (emprunterLivre(String isbn, int idMembre)).
- **Enregistrer un retour** (retournerLivre(int idEmprunt)).
- **Afficher tous les emprunts** (getTousLesEmprunts()).
- **Afficher les emprunts en cours** (getEmpruntsEnCours()).
- **Afficher les emprunts en retard** (getEmpruntsEnRetard()).

- **Se déconnecter**

- Comme pour un membre, l'employé peut se déconnecter.

❖ **Fonctionnalités Supplémentaires**

La méthode afficherLivres(List<Livre> livres)

- Affiche les informations des livres disponibles.
- Gère le cas où aucun livre ne correspond à la recherche.

VIII. Classe principale pour lancer l'application

```

1 package bibliotheques;
2 import bibliotheque.ui.InterfaceConsole;
3
4 public class Main {
5     public static InterfaceConsole interfaces;
6
7     public static void main(String[] args) {
8         interfaces = new InterfaceConsole();
9         interfaces.demarrer();
10
11     }
12 }

```

Explication

Cette classe Main est le **point d'entrée** du programme. Son rôle est de **lancer l'application** et d'afficher l'interface utilisateur en ligne de commande.

- ❖ **package bibliotheques;** → Définit le package contenant cette classe.
- ❖ **import bibliotheque.ui.InterfaceConsole;** → Importe la classe InterfaceConsole qui gère l'interaction avec l'utilisateur.
- ❖ **Déclaration des Variables : interfaces** est un objet de type InterfaceConsole, utilisé pour gérer l'interface en ligne de commande.
- ❖ Méthode main ()
 - **new InterfaceConsole()** → Crée une instance de l'interface console.
 - **interfaces.demarrer();** → Lance l'interface en appelant la méthode demarrer(), qui va gérer la boucle d'interaction avec l'utilisateur.

IX. Test de l'application

- **Employé**

On a débuté en nous connectant en tant qu'employé pour ainsi tester les différentes fonctionnalités du menu employé.

Toujours dans le menu employé, on a ajouté un nouveau livre et on a aussi inscrit un nouveau membre. Ensuite, on a enregistré un emprunt au nouveau membre.

```
Problems Javadoc Declaration Console X Coverage
<terminated> Main [Java Application] /Users/thegenius/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.ma
=== SYSTÈME DE GESTION DE BIBLIOTHÈQUE ===

=== AUTHENTIFICATION ===
1. Se connecter en tant que membre
2. Se connecter en tant qu'employé
Votre choix : 2
Entrez votre ID employé : 1
Bienvenue, Michel Dubois !

Voulez-vous continuer ? (O/N)
0

=== MENU EMPLOYÉ ===
1. Gérer les livres
2. Gérer les membres
3. Gérer les emprunts
4. Se déconnecter
Votre choix : 2

=== GESTION DES MEMBRES ===
1. Inscrire un nouveau membre
2. Supprimer un membre
3. Consulter tous les membres
4. Rechercher un membre par ID
Votre choix : 1
Nom : Mbaye
Prénom : Abdoulaye
Email : ablaye@gmail.com
Téléphone : 771234567
Adresse : mermoz
Membre inscrit avec succès ! ID : 3
```

```
Problems Javadoc Declaration Console X Coverage
<terminated> Main [Java Application] /Users/thegenius/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.fu
Voulez-vous continuer ? (O/N)
0

=== MENU EMPLOYÉ ===
1. Gérer les livres
2. Gérer les membres
3. Gérer les emprunts
4. Se déconnecter
Votre choix : 2

=== GESTION DES MEMBRES ===
1. Inscrire un nouveau membre
2. Supprimer un membre
3. Consulter tous les membres
4. Rechercher un membre par ID
Votre choix : 3

=== LISTE DES MEMBRES ===
Membre #1: Jean Dupont (jean.dupont@email.com)
Membre #2: Sophie Martin (sophie.martin@email.com)
Membre #3: Abdoulaye Mbaye (ablaye@gmail.com)

Voulez-vous continuer ? (O/N)
0

=== MENU EMPLOYÉ ===
1. Gérer les livres
2. Gérer les membres
3. Gérer les emprunts
4. Se déconnecter
Votre choix : 1
```

```
Problems Javadoc Declaration Console X Coverage
<terminated> Main [Java Application] /Users/thegenius/.p2/pool/plugins/org.eclipse.justj.openjdk.h
=== GESTION DES LIVRES ===
1. Ajouter un livre
2. Supprimer un livre
3. Mettre à jour un livre
4. Consulter tous les livres
5. Rechercher un livre par ISBN
Votre choix : 1
ISBN : 123
Titre : Mamacito
Auteur : Pokotio
Éditeur : Caba
Année de publication : 2024
Nombre d'exemplaires : 1
Livre ajouté avec succès !

Voulez-vous continuer ? (O/N)
0

=== MENU EMPLOYÉ ===
1. Gérer les livres
2. Gérer les membres
3. Gérer les emprunts
4. Se déconnecter
Votre choix : 1

=== GESTION DES LIVRES ===
1. Ajouter un livre
2. Supprimer un livre
3. Mettre à jour un livre
4. Consulter tous les livres
5. Rechercher un livre par ISBN
Votre choix : 4
```



```
Problems Javadoc Declaration Console X Coverage
<terminated> Main [Java Application] /Users/thenigenius/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.x86_64_17.0.13.v20241023-1126/jre/bin/java
=== LISTE DES LIVRES ===
Mamacito par Pokotio (ISBN: 123) - 1/1 disponibles
Le Petit Prince par Antoine de Saint-Exupéry (ISBN: 978-2-7654-1005-3) - 5/5 disponibles
1984 par George Orwell (ISBN: 978-2-253-06541-0) - 2/2 disponibles
L'Étranger par Albert Camus (ISBN: 978-2-07-036822-8) - 3/3 disponibles

Voulez-vous continuer ? (O/N)
0

=== MENU EMPLOYÉ ===
1. Gérer les livres
2. Gérer les membres
3. Gérer les emprunts
4. Se déconnecter
Votre choix : 3

=== GESTION DES EMPRUNTS ===
1. Enregistrer un nouvel emprunt
2. Enregistrer un retour
3. Consulter tous les emprunts
4. Consulter les emprunts en cours
5. Consulter les emprunts en retard
Votre choix : 1
ID du membre : 3
ISBN du livre :
123
Emprunt enregistré avec succès ! ID : 1
```

```
Problems Javadoc Declaration Console X Coverage
<terminated> Main [Java Application] /Users/thenigenius/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.x86_64_17.0.13.v20241023-1126/jre/bin/java
Voulez-vous continuer ? (O/N)
0

=== MENU EMPLOYÉ ===
1. Gérer les livres
2. Gérer les membres
3. Gérer les emprunts
4. Se déconnecter
Votre choix : 1

=== GESTION DES LIVRES ===
1. Ajouter un livre
2. Supprimer un livre
3. Mettre à jour un livre
4. Consulter tous les livres
5. Rechercher un livre par ISBN
Votre choix : 4

=== LISTE DES LIVRES ===
Mamacito par Pokotio (ISBN: 123) - 0/1 disponibles
Le Petit Prince par Antoine de Saint-Exupéry (ISBN: 978-2-7654-1005-3) - 5/5 disponibles
1984 par George Orwell (ISBN: 978-2-253-06541-0) - 2/2 disponibles
L'Étranger par Albert Camus (ISBN: 978-2-07-036822-8) - 3/3 disponibles

Voulez-vous continuer ? (O/N)
0

=== MENU EMPLOYÉ ===
1. Gérer les livres
2. Gérer les membres
3. Gérer les emprunts
4. Se déconnecter
Votre choix : 4
Vous êtes déconnecté.

Voulez-vous continuer ? (O/N)
0
```

- **Membre**

Par la suite, on se connecte en tant que membre pour tester les différentes fonctionnalités. On voit bien que l'emprunt à bien était enregistré.

```
Problems Javadoc Declaration Console X Coverage
<terminated> Main [Java Application] /Users/thegenius/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.x86_64_17.0.13.v20241023-1126/jre/bin/ja

Voulez-vous continuer ? (O/N)
0

=== AUTHENTIFICATION ===
1. Se connecter en tant que membre
2. Se connecter en tant qu'employé
Votre choix : 1
Entrez votre ID membre : 3
Bienvenue, Abdoulaye Mbaye !

Voulez-vous continuer ? (O/N)
0

=== MENU MEMBRE ===
1. Consulter les livres disponibles
2. Rechercher un livre par titre
3. Rechercher un livre par auteur
4. Consulter mes emprunts
5. Se déconnecter
Votre choix : 1

=== LISTE DES LIVRES ===
Mamacito par Pokotio (ISBN: 123) - 0/1 disponibles
Le Petit Prince par Antoine de Saint-Exupéry (ISBN: 978-2-7654-1005-3) - 5/5 disponibles
1984 par George Orwell (ISBN: 978-2-253-06541-0) - 2/2 disponibles
L'Étranger par Albert Camus (ISBN: 978-2-07-036822-8) - 3/3 disponibles

Voulez-vous continuer ? (O/N)
0

=== MENU MEMBRE ===
1. Consulter les livres disponibles
2. Rechercher un livre par titre
3. Rechercher un livre par auteur
4. Consulter mes emprunts
5. Se déconnecter
Votre choix : 4

=== MES EMPRUNTS ===
Emprunt #1: Mamacito par Abdoulaye Mbaye - À retourner avant le Tue Apr 15 02:17:33 GMT 2025

Voulez-vous continuer ? (O/N)
N
Au revoir !
```

Conclusion

La mise en place de ce **système de gestion de bibliothèque** apportera une amélioration significative en termes d'efficacité et d'organisation. Il permettra un suivi rigoureux des ressources et facilitera les interactions entre les membres et les administrateurs. En assurant une bonne structuration des entités et une gestion sécurisée des données, ce projet garantira un outil fiable et évolutif, capable de répondre aux besoins croissants des bibliothèques modernes.

FIN